

UNIDAD II

Análisis Semántico

2.1 Introducción

Analizador Semántico.

- ❖ Verifica que el significado de las construcciones del lenguaje tengan sentido.
- ❖ Tareas del analizador semántico:
 - 1) Comprobación de Tipos.
 - 2) Comprobación de parámetros.
 - 3) Generación de código intermedio.

Ejemplo:

```

A : float ;
B : string ;
.
.
.
A := B * 5 ;

```

Analizador Léxico

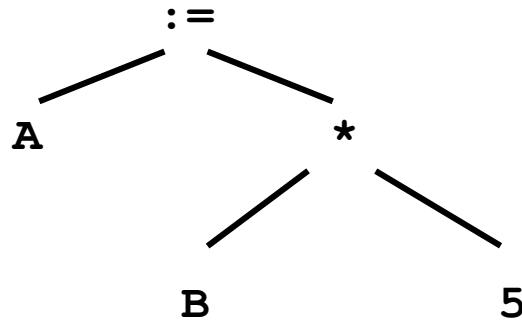
```

id1 : float ;
id2 : string ;
.
.
.
id1 := id2 op num3 ;

```

TABLA DE SÍMBOLOS

	Lexema	Complex	Tipo	...
1	A	id	?	
2	B	id	?	
3	5	num	?	

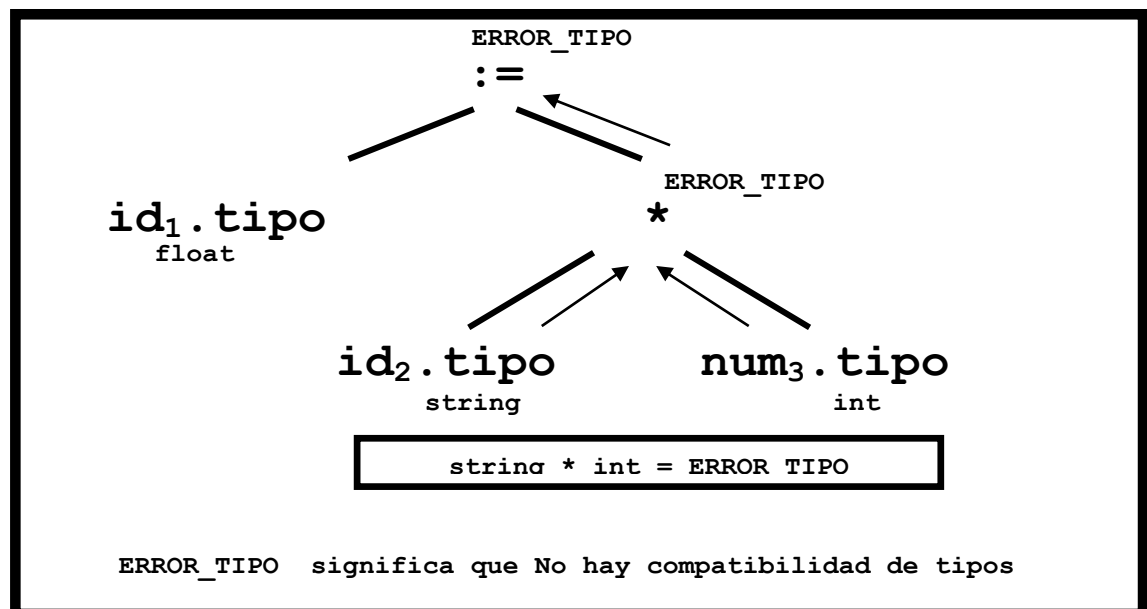
Analizador SintácticoAnalizador Semántico

Hasta aquí la entrada es léxica y sintácticamente válida, ahora se analiza desde el punto de vista semántico. El analizador semántico utilizará un mecanismo para registrar en la Tabla de Símbolos el tipo de dato con el que se declararon las variables:

TABLA DE SÍMBOLOS

	Lexema	Complex	Tipo	...
1	A	id	float	
2	B	id	string	
3	5	num	int	

Con dicha información en la Tabla de Símbolos y aplicando una serie de reglas de comprobación se detectará que al operar la multiplicación de un tipo **string** por un tipo **int** no tiene sentido. Una situación como ésta se califica como **ERROR_TIPO**, el cual nos indica que en esa parte de la sentencia hay un problema de compatibilidad de tipos, en consecuencia la sentencia de asignación completa se califica de igual manera como **ERROR_TIPO**.



Atributos y reglas semánticas

Un compilador puede necesitar tener en cuenta muchas características además del código generado para la construcción de entrada. Para realizar un análisis semántico a cada construcción del lenguaje se le asocia una serie de atributos así como de acciones o reglas semánticas.

Un *atributo* es información asociada a un terminal o a un no-terminal, y puede representar una cadena o una posición de memoria.

Regla Semántica: Acción o conjunto de acciones(algoritmo) para calcular el valor de los atributos y para realizar las comprobaciones de tipos.

Los analizadores semánticos se construyen asociando una serie de atributos y acciones o reglas semánticas a cada construcción del lenguaje.

Dos formas de asociarlo:

- 1) Definición dirigida por la sintaxis.
- 2) Esquema de traducción.

En ambos casos es útil definir un conjunto de atributos a los símbolos gramaticales del lenguaje.

Dos tipos de atributos:

- 1) Sintetizados.
- 2) Heredados.

2.2 Definiciones dirigidas por la sintaxis ²

**Definición
dirigida por
la sintaxis** = **Gramática** + **Reglas
Semánticas**

El concepto *Definición dirigida por la sintaxis* se utiliza para especificar las traducciones para las construcciones del lenguaje(sentencias) en función de *atributos asociados* con sus componentes sintácticos.

Una definición dirigida por la sintaxis usa una gramática para especificar la estructura sintáctica de la entrada. A cada símbolo de la gramática se le asocia un conjunto de atributos y a cada producción un conjunto de *reglas semánticas*, para calcular los valores de los atributos asociados con los símbolos que aparecen en esa producción.

Las definiciones dirigidas por la sintaxis no solo se utilizan para la comprobación de tipos, se puede usar para transformar una sentencia o cadena de entrada a cualquier forma de salida.

² Libro del Dragón, p.288

Ejemplo: Definición dirigida por la sintaxis para traducir expresiones infijas a postfijas:

PRODUCCION	REGLA SEMÁNTICA
$\text{expr} \rightarrow \text{expr}_1 + \text{término}$	$\text{expr.t} := \text{expr}_1.\text{t} \ \ \text{término.t} \ \ '+'$
$\text{expr} \rightarrow \text{expr}_1 - \text{término}$	$\text{expr.t} := \text{expr}_1.\text{t} \ \ \text{término.t} \ \ '-'$
$\text{expr} \rightarrow \text{término}$	$\text{expr.t} := \text{término.t}$
$\text{termino} \rightarrow 0$	$\text{termino.t} := '0'$
$\text{termino} \rightarrow 1$	$\text{termino.t} := '1'$
.....	
$\text{termino} \rightarrow 9$	$\text{termino.t} := '9'$

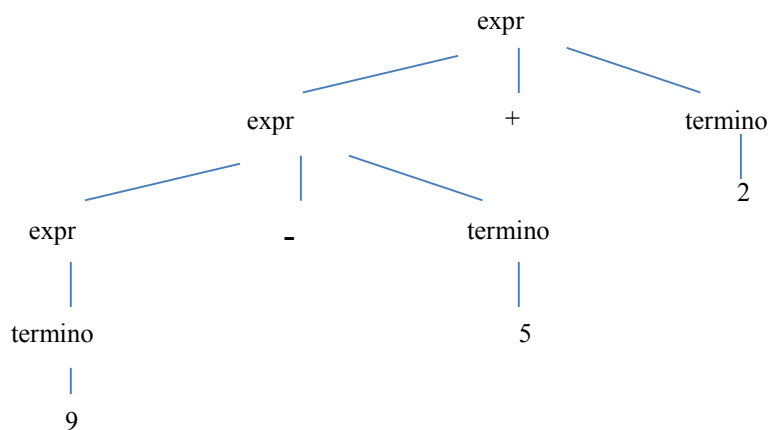
Usando la Definición Dirigida por la Sintaxis anterior analizar la siguiente expresion infija: $9 - 5 + 2$

Es este caso la expresion postfija esperada es:

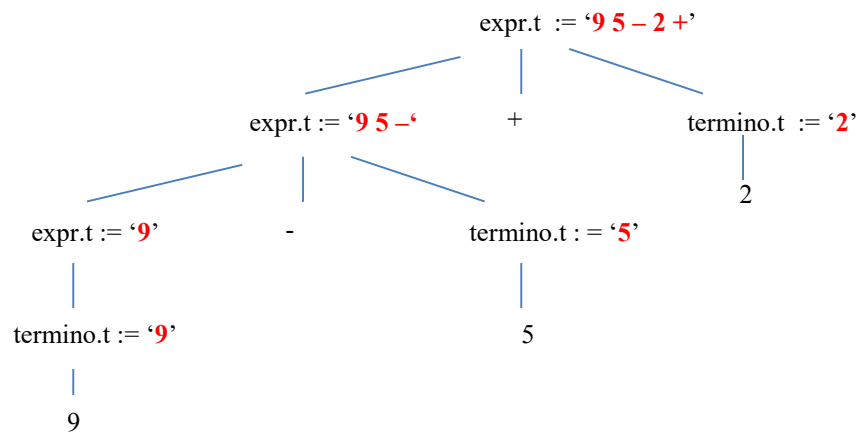
Postfija : operando operando operador $9 \ 5 \ - \ 2 \ +$

LEXICO: $9 \ - \ 5 \ + \ 2$

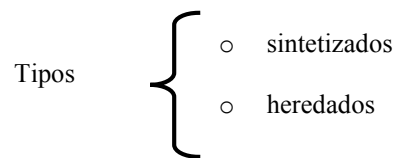
SINTACTICO:



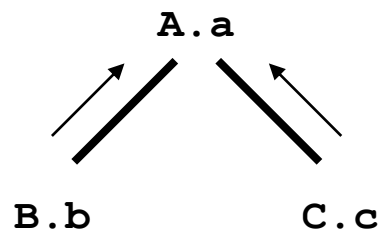
SEMANTICO: Se realiza un recorrido en profundidad de izquierda a derecha, visitando todos los nodos del árbol y aplicando las reglas semánticas cuando sea posible.



Tipos de atributos³.



❖ **Sintetizados.** Su valor esta en función de los atributos en nodos *hijos*.

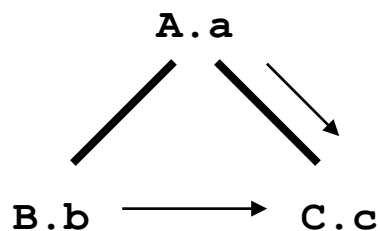


Donde:

a,b,c son atributos
 si $a := f(b \text{ y/o } c)$
 .
 . . **a** es sintetizado

³ Libro del Dragón, p.290

- ❖ **Heredados.** Su valor esta en función de los atributos en nodos *padres* y/o *hermanos*.



Donde:

a,b,c son atributos
 si $c := f(b \text{ y/o } a)$
 . . . **c** es heredado

Definición dirigida por la sintáxis para la declaración de identificadores integer y real

PRODUCCION

$D \rightarrow T L$

$T \rightarrow \text{int}$

$T \rightarrow \text{real}$

$L \rightarrow L_1, \text{id}$

$L \rightarrow \text{id}$

REGLA SEMÁNTICA

$L.h := T.tipo$

$T.tipo := \text{'integer'}$

$T.tipo := \text{'real'}$

$L_1.h := L.h$
 $\text{anadetipo}(\text{id.entrada}, L.h)$

$\text{anadetipo}(\text{id.entrada}, L.h)$

Atributo

$L.h$

$T.tipo$

Tipo de Atributo

heredado

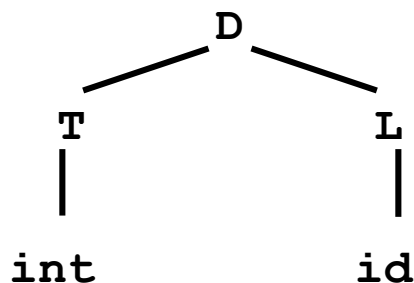
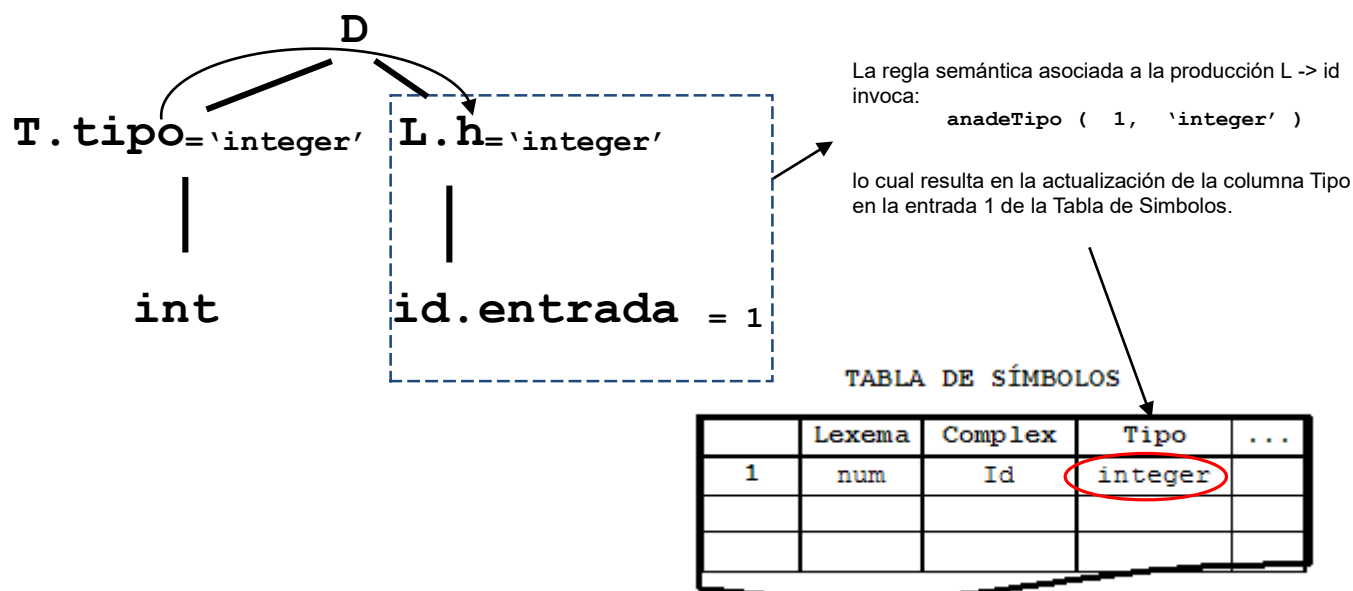
sintetizado

Ejemplo: Analizar la siguiente sentencia de entrada:

int num

Analizador Léxico

int id

Analizador SintácticoAnalizador Semántico**Ejercicios:**

1. Similar al ejemplo anterior, evaluar la siguiente sentencia de entrada:

real a, b, c

2. La siguiente gramática genera expresiones separadas mediante el operador aritmético "+" a constantes enteras y reales. Cuando se suman 2 enteros el tipo obtenido es un entero de lo contrario es real:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow \text{num.num} \mid \text{num}$$

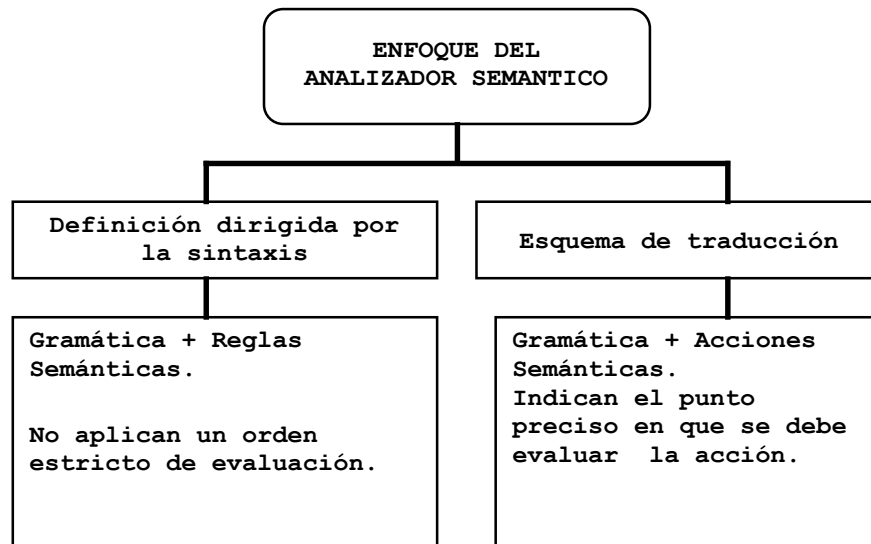
Dar una Definición Dirigida por la Sintaxis para determinar el tipo de cada expresión.

3. Dar una **Definición Dirigida por la Sintaxis** para registrar en la tabla de símbolos el tipo de dato de los identificadores cuando son declarados en lenguaje Pascal.

G r a m á t i c a

declaraciones	→ var lista_identificadores : tipo ; declaraciones
declaraciones	→ ϵ
tipo	→ tipo_estandar
tipo_estandar	→ integer
tipo_estandar	→ real
lista_identificadores	→ id lista_identificadores'
lista_identificadores'	→ , id lista_identificadores'
lista_identificadores'	→ ϵ

2.3 Esquemas de traducción⁴.



Un *esquema de traducción* es una gramática independiente de contexto en la que se asocian atributos con los símbolos gramaticales y se insertan acciones semánticas encerradas entre llaves dentro de los lados derechos de las producciones. Los *esquemas de traducción* son una notación útil para *especificar la traducción* durante el análisis sintáctico.

Ejemplo: El siguiente Esquema de Traducción transforma expresiones infijas con suma y resta en expresiones postfijas :

$$E \rightarrow T \ R$$

$$R \rightarrow \text{oparit } T \{ \text{printf}(\text{oparit.lexema}) \} R_1 \mid \varepsilon$$

$$T \rightarrow \text{num} \{ \text{printf}(\text{num.lexema}) \}$$

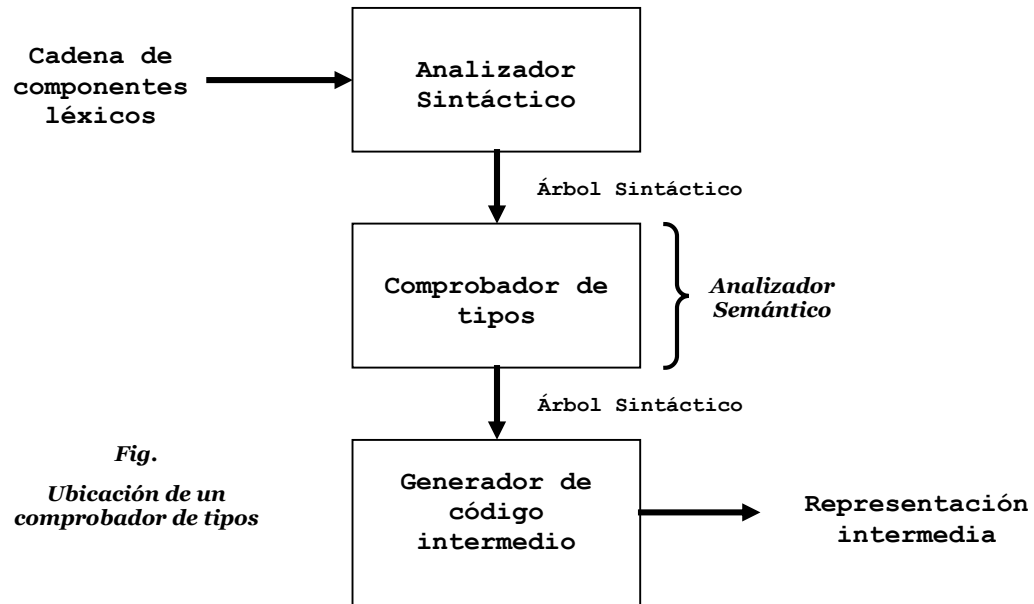
Las acciones semánticas van encerradas entre llaves y se consideran como símbolos terminales, lo cual es conveniente para establecer cuando se deben ejecutar las acciones. Es decir, los *esquemas de traducción* establecen el orden preciso en que se deben evaluar las acciones semánticas.

Ejercicio. Diseñar como **Esquema de Traducción** el registro del tipo de dato en la tabla de símbolos de los identificadores cuando son declarados en lenguaje Pascal.

⁴ Libro del Dragón, p.307

2.4 Comprobación de tipos.

Un *Comprobador de Tipos* se asegura de que el tipo de una construcción coincida con el previsto en su contexto. Por ejemplo, el operador “mod” en Pascal exige operandoos de tipo entero, de igual manera debe asegurarse de que la indización que se haga sobre una matriz sea con un índice entero, y de que a una función definida por el usuario se aplique el número y tipo correcto de argumentos.



Un compilador debe comprobar si el programa fuente sigue las convenciones sintácticas y semánticas del lenguaje fuente. Dicha comprobación puede ser de dos tipos:

1. *Estática*. La comprobación ocurre al compilador.
2. *Dinámica*. La comprobación se realiza al ejecutar el programa

En la comprobación *Estática* existen las siguientes ventajas:

- i. Facilita la depuración de programas ya que verifica los errores posibles.
- ii. No requiere guardar toda la información acerca de los objetos de datos.
- iii. A estos lenguajes se les conoce como “fuertemente tipados”, ejemplo: Java, C#, C++, Pascal, etc.

En la comprobación *Dinámica* su ventaja principal es la flexibilidad que permite en el diseño de lenguaje ya que:

- i. No requiere una definición previa del tipo de dato para las variables.
- ii. El tipo de dato asociado al nombre de la variable puede cambiar durante su ejecución.
- iii. Lenguajes como JavaScript y Python usan comprobación dinámica.

2.5 Expresiones de tipo⁵.

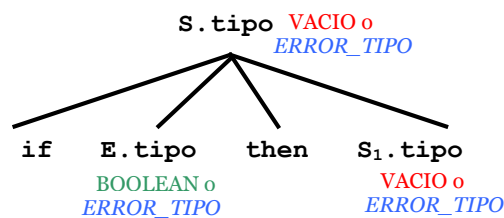
Es o bien un tipo básico (booleano,char,integer,real) o una expresión que se forma aplicando un operador llamado constructor de tipos a otras expresiones de tipo

⁵ Libro del Dragón, p. 357

1. Tipo Básico.

Un tipo básico es por sí solo una expresión de tipo; al conjunto de tipos básicos se agregan el tipo básico especial *error_tipo*, el cual señala el error durante la comprobación de tipos. Además un tipo básico *vacío* que indica la ausencia de error y permite que se comprueben las proposiciones que no requieren un tipo de dato específico.

Ejemplo. En la siguiente producción **S** representa una sentencia if-then, el atributo **S.tipo** solo podría tomar el valor de *vacío* si los tipos de dato de los elementos sintácticos que participan en la sentencia if-then están correctos, de otra forma **S.tipo** tomaría el valor de *error_tipo*.

$$S \rightarrow \text{if } E \text{ then } S_1$$


S.tipo solo puede valer VACIO si **E.tipo** es BOOLEAN y **S1.tipo** es VACIO.

2. Constructor de tipos.

Un *constructor de tipos* aplicado a expresiones de tipo es también una expresión de tipos. Los constructores de tipos incluyen:

i. MATRICES:

Si **T** es una expresión de tipo, entonces:

array (I , T)

es una expresión de tipo que indica el tipo de una matriz con elementos de tipo **T** y un conjunto de índices **I**, donde **I** es un rango de enteros.

Ejemplo:

var **A** : array [1...10] of integer

¿Cuál es la expresión de tipo para el identificador **A** ?

array (1...10 , integer)

TABLA DE SÍMBOLOS

	Lexema	Complex	Tipo	...
1	A	id	array(1...10 , integer)	

ii. PRODUCTOS:

Si T_1 y T_2 son expresiones de tipo entonces su producto cartesiano:

$$T_1 \times T_2$$

es también una expresión de tipo. Este tipo de constructor se utiliza para definir los tipos de elementos individuales de un tipo del mayor nivel.

Ejemplo:

```
var A : array [ 1...10 ] of integer
```

su expresión de tipo como producto sería:

```
integer x integer x ... ( 10 veces )
```

iii. REGISTROS:

El constructor de tipos *record* se aplicará a una tabla formada con nombres de campos y tipos de datos.

```
record ( T )
```

Ejemplo:

```
type fila = record
    direccion : integer;
    lexema    : array[1...15] of char;
end;
```

La expresión de tipo para la declaración del tipo *fila* es:

```
record (integer x array ( 1...15 , char ) )
```

iv. APUNTADES

Si T es una expresión de tipo, entonces:

```
pointer ( T )
```

es una expresión de tipo que indica el tipo "apuntador a un objeto de tipo T ".

Ejemplo:

```
var i : ^ integer
```

La expresión de tipo para *i* sería:

```
pointer ( integer )
```

v. FUNCIONES.

Indica el tipo de una función que transforma un dominio de tipo D a un rango tipo R . La expresión de tipo:

$$D \rightarrow R$$

indicará el tipo de función. Por ejemplo, la función predefinida "mod" de pascal tiene un dominio de tipo:

int x int

es decir, un par de enteros y rango de tipo:

int

entonces la expresión de tipo completa es:

int x int $\rightarrow$ int

Ejemplo:

```
function miFuncion ( a , b : char ) : ^ integer
char x char  $\rightarrow$  pointer ( integer )
```

Ejercicio. Dar la expresión de tipo de los siguientes identificadores en negrita:

- a) int ***miArreglo** [10]
- b) void **func** (char *c, int i, float f)
- c) byte **triDi** [5][10][15]
- d) class **miClase**
{
 int i;
 bool b;
 double d [20];
 void metodo1 ();
 bool metodo2 (double d);
}

2.6 Comprobador de tipos de ejemplo.

Este, esta diseñado como un esquema de traducción que sintetiza el tipo de cada expresión:

```

P → D ; S
D → D ; D
D → id : T {anadetipo(id.entrada), T.tipo;} ①
T → char {T.tipo:=char;} ②
T → integer {T.tipo:=integer;} ③
T → ↑ T1 {T.tipo:=pointer(T1.tipo);} ④
T → array[num] of T1 {T.tipo:=array (1..num.lexema, T1.tipo);} ⑤
S → id:=E {S.tipo:=if buscatipo(id.entrada)==E.tipo then VACIO;
                                     else ERROR_TIPO;} ⑥
S → if B then S1 {S.tipo:=if B.tipo==BOOLEAN AND S1.tipo==VACIO then
                                     VACIO;
                                     else ERROR_TIPO;} ⑦
S → while B do S1 {S.tipo:=if B.tipo==BOOLEAN AND S1.tipo==VACIO then
                                     VACIO;
                                     else ERROR_TIPO;} ⑧

S → S1 ; S2 {S.tipo:=if S1.tipo== VACIO AND S2.tipo==VACIO then
                                     VACIO;
                                     else ERROR_TIPO;} ⑨
E → literal {E.tipo:=char;} ⑩
E → num {E.tipo:=integer;} ⑪
E → id {E.tipo:= buscatipo(id.entrada);} ⑫
E → E1 mod E2 {E.tipo:=if E1.tipo==integer AND E2.tipo==integer then
                                     integer; ⑬
                                     else ERROR_TIPO;}

E → M {E.tipo := M.tipo } ⑭
E1 → E2 + M {E1.tipo := if E2.tipo = integer AND M.tipo = integer then
                                     Integer
                                     Else ERROR_TIPO } ⑮
M → F {M.tipo := F.tipo } ⑯
M1 → M2 * F {M1.tipo := if M2.tipo = integer AND F.tipo = integer then
                                     Integer
                                     Else ERROR_TIPO } ⑰
F → (E) {F.tipo := E.tipo } ⑱
F → id {F.tipo := buscaTipo ( id.entrada ) } ⑲
F → num {F.tipo := integer } ⑳
B → E1 oprel E2 {B.tipo := if E1.tipo == E2.tipo then BOOLEAN
                                     else ERROR_TIPO } ㉑

```

Ejemplo: Hacer la comprobación de tipo del siguiente programa

```

x : char ;
y : integer ;
x := "Hola muchachos" ;
if y > 5 then
    y := x

```

Analizador Léxico

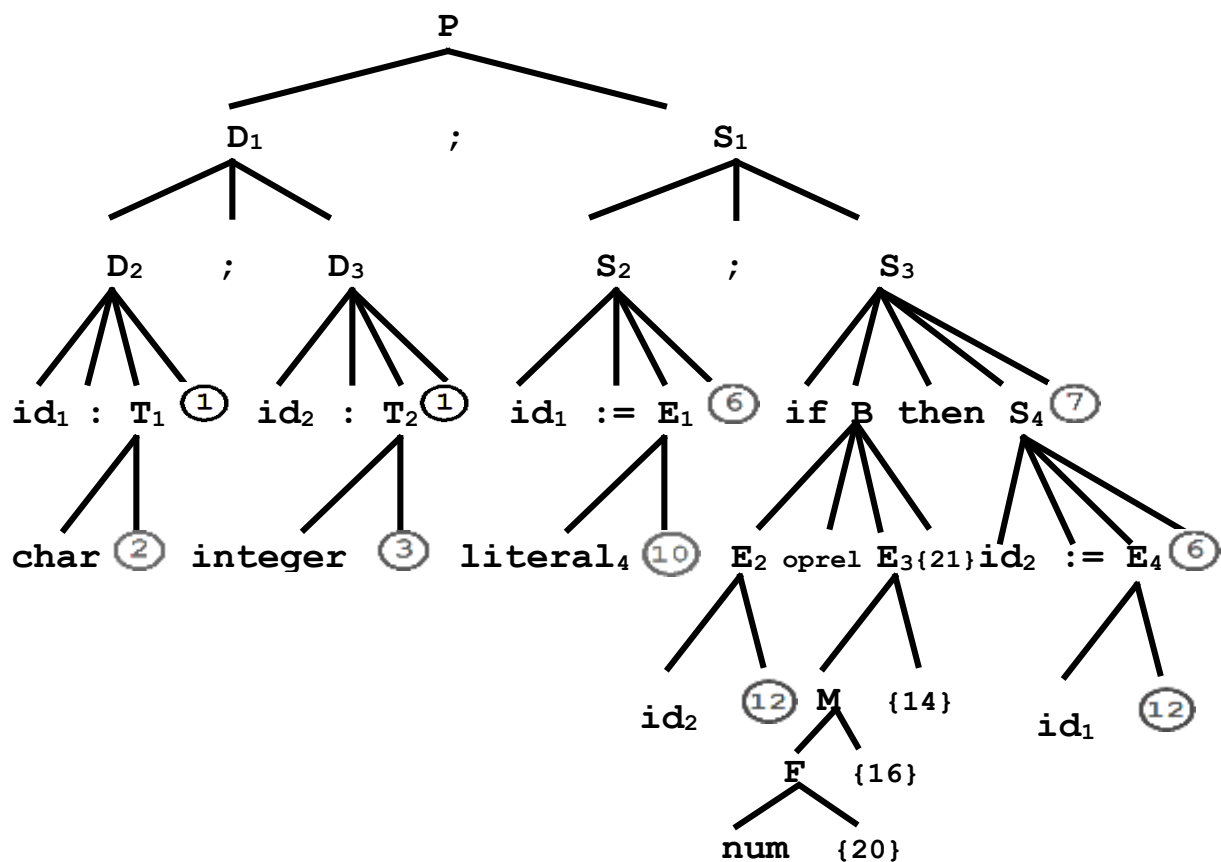
```

id1 : char;
id2 : integer;
id1 := literal3;
if id2 oprel num4 then
    id2 := id1;

```

	Lexema	Complex	Tipo	...
1	x	id		
2	y	id		
3	5	num		
4	"Hola muchachos"	literal		

Analizador Sintáctico – Semántico



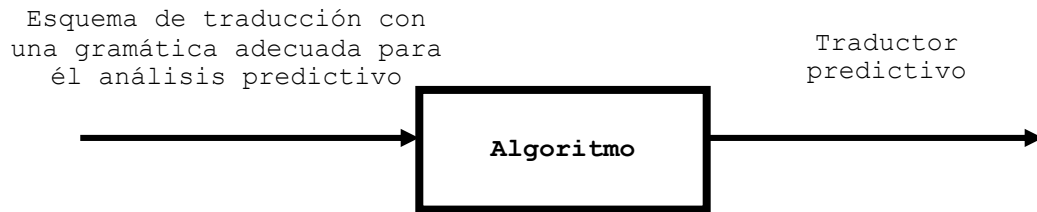
Recorrido del árbol (Comprobador de Tipos)

ATRIBUTOS	
Símbolo	Tipo
P	
D ₁	
D ₂	
D ₃	
T ₁	char
T ₂	integer
S ₁	ERROR_TIPO
S ₂	VACIO
S ₃	ERROR_TIPO
E ₁	char
S ₄	ERROR_TIPO
E ₄	char
B	BOOLEAN
E ₂	integer
E ₃	integer
M	integer
F	integer

TABLA DE SÍMBOLOS				
	Lexema	Complex	Tipo	...
1	x	id	char	
2	y	id	integer	
3	5	num	Integer	
4	"Hola muchachos"	literal	char	

. . **ERROR_TIPO** (No hay compatibilidad de tipos): Al determinar el valor de S₄.tipo es dónde se detecta el error de tipos, que corresponde a la asignación `y := x` porque son de tipo diferente y el lenguaje obliga a que sean tipos iguales para estar correcto.

2.7 Diseño de un traductor predictivo



Algoritmo planteado en la bibliografía del Libro del Dragón :

1. Para cada no-terminal A constrúyase una función que tenga un parámetro formal para cada atributo heredado de A y que devuelva los atributos sintetizados de A. La función para A tiene una variable local para cada atributo de cada símbolo gramatical que aparezca en una producción para A.
2. El código para el no-terminal A decide que producción utiliza basándose en el símbolo en curso de entrada.
3. El código asociado con cada producción hace lo siguiente:
 - a) Para el símbolo terminal X con atributo sintetizado x, guárdese el valor de x en la variable declarada X.x después emparéjese X.
 - b) Para el no-terminal B genérese una asignación

$$c := B (b_1, b_2, \dots b_n)$$

Donde: $b_1, b_2, \dots b_n$ son las variables para los atributos heredados de B.
 c es la variable para el atributo sintetizado de B.

- c) Para el caso de una acción semántica cópiese el código de la acción dentro del analizador sintáctico, sustituyendo cada referencia a un atributo por la variable correspondiente a dicho atributo.

Interpretación del algoritmo aplicado a la implementación en un lenguaje orientado a objetos:

1. Definir una clase llamada **Atributos** para concentrar todos los atributos a los que se hace referencia en las acciones semánticas, excepto los atributos entrada, lexema y complex.

Ejemplo:

```

public class Atributos
{
    String tipo;
    String h;
    ...
}
  
```

Nota: En la clase atributos se incluyen tanto atributos heredados como sintetizados.

2. Modificar la firma de cada método que corresponde a los *procedures*, para definir un argumento de la clase Atributos donde se pasarán los atributos tanto heredados como sintetizados de ese símbolo no-terminal.

Ejemplo:

```

private void SENTENCIA ( Atributos SENTENCIA )
  
```

Los atributos sintetizados que viajan en el argumento sirven como parametros de entrada.
 Los atributos heredados que vienen en el argumento serán parametros de salida.

3. En cada procedure declarar una variable local de la clase *Atributos* para cada símbolo gramatical que aparezca en el lado derecho de la producción y que tenga definidos atributos.

Para los símbolos gramaticales *id*, *num*, *num.num* y *literal* se deben declarar de la clase *Linea_BE*, que significa Linea del Buffer de Entrada.

Ejemplo: Sea la producción $A \rightarrow B \text{ id } \{1\} C \{2\}$

y suponiendo que las acciones semánticas {1} y {2} son las siguientes:

```
{1}: C.h := buscaTipo ( id.entrada )

{2}: A.tipo := if B.tipo == VACIO AND C.tipo == VACIO then
           VACIO
        else
           ERROR_TIPO
```

Las variables locales que se declararían en el *procedure* de A son:

```
private void A ( Atributos A )
{
    // Variables locales
    Atributos B = new Atributos ();
    Atributos C = new Atributos ();
    Linea_BE id = new Linea_BE ();
    ...
}
```

4. Implementar el lado derecho de la producción (retomando el ejemplo del punto 3 anterior):

- a) Si es un símbolo **terminal** con atributos, **salvar** los atributos en su variable local **antes** de emparejarlo.

Ejemplo:

```
id = cmp.be.preAnalisis; // se salvan los atributos de id
emparejar ( "id" );
```

- b) Si es un símbolo **no-terminal** se invoca su procedimiento pasando como argumento la variable local que tiene sus atributos.

Ejemplo:

```
B ( B );
...
C ( C );
```

- c) Si es una acción semántica transcribir a código la acción semántica en el punto donde está insertada, sustituyendo las referencias a atributos por la variable local correspondiente.

Ejemplo:

```
B ( B );
id = cmp.be.preAnalisis;
emparejar ( "id" );
```

```

// Acción semántica 1
C.h = cmp.ts.buscaTipo ( id.entrada );
// Fin accion semantica 1

C ( C );

// Acción semántica 2
if ( B.tipo.equals ( VACIO ) && C.tipo.equals ( VACIO ) )
    A.tipo = VACIO;
else {
    A.tipo = ERROR_TIPO;
    cmp.me.error ( Compilador.ERR_SEMANTICO, "descripcion del error" );
}
// Fin accion semantica 2

```

Ejemplo 1: Implementar la siguiente producción que sirve para declarar un identificador y su correspondiente tipo de dato. La acción semántica 4 sirve para registrar el tipo de dato del identificador en la tabla de símbolos.

```

V -> id : T {4}

{4} = anadeTipo ( id.entrada, T.tipo )
      V.tipo := VACIO

```

El pseudocódigo que implementa el procedimiento del símbolo V sería:

```

private void V ( Atributos _V )
{
    Atributos _T;    // Atributos del símbolo T
    Linea_BE id;     // Atributos del símbolo id

    if ( preAnalisis.equals ( "id" ) ) {
        id = cmp.be.preAnalisis;    // Salva los atributos del símbolo id
        emparejar ( "id" );
        emparejar ( ":" );
        T ( _T );

        // Acción semántica 4 -----
        anadeTipo ( id.entrada, _T.tipo );
        V.tipo = VACIO;
        // Fin acción semántica 4 -----
    }
    else
        error ( "..." );    // error sintáctico
}

```

Ejemplo 2: Considere la siguiente producción que genera la sentencia SQL

```

S -> DELETE FROM id WHERE C {1}

{1} : { S.tipo := if buscaTipo ( id.entrada ) == "table" AND
              C.tipo == "boolean" then
                  VACIO
              else
                  ERROR_TIPO
          }

```

El pseudocódigo que implementa el procedimiento del símbolo S sería:

```
procedure S ( Atributos _S )
begin
  Atributos _C      // Atributos del símbolo C
  Integer id.entrada // Atributos del símbolo id

  If preAnálisis == "DELETE" then
    Begin
      emparejar ( "DELETE" )
      emparejar ( "FROM" )

      // Salvamos los atributos de id antes de emparejarlo.
      // Solo salvamos el atributo entrada porque es el único que
      // se utiliza de id en las acciones semánticas
      // de esta producción.
      id.entrada := preAnálisis.entrada
      emparejar ( "id" )
      emparejar ( "WHERE" )

      // Invocamos el procedimiento de C pasándole sus atributos
      // heredados y
      // después de la llamada nos devolverá sus atributos sintetizados
      C ( _C )

      // Acción semántica 1
      If buscaTipo ( id.entrada ) = "table" AND _C.tipo = "boolean" then
        begin
          _S.tipo := VACIO
        end
      else
        begin
          _S.tipo := ERROR_TIPO
          error ( ) // Error Semántico
        end
      //Fin de la Acción semántica 1
    End
  Else
    error ( ) // Error Sintáctico
end;
```

EJERCICIO. Implementar en Java el siguiente comprobador de tipos

P → V C {1}
 V → id : T {2} V₁ {3} | empty {4}
 T → entero {5} | real {6} | caracter {7}
 C → inicio S fin {8}
 S → id opasig E {9}
 E → num {10} | num.num {11}

No. Accion	Accion Semántica
1	P.tipo := if V.tipo == VACIO and C.tipo == VACIO then VACIO else ERROR_TIPO
2	V ₁ .h := if buscaTipo (id.entrada) == "" then begin añadeTipo (id.entrada, T.tipo) VACIO end else ERROR_TIPO
3	V.tipo := if V ₁ .h == VACIO and V ₁ .tipo == VACIO then VACIO else ERROR_TIPO
4	V.tipo := VACIO
5	T.tipo := "integer"
6	T.tipo := "real"
7	T.tipo := "caracter"
8	C.tipo := S.tipo
9	S.tipo := if buscaTipo (id.entrada) == E.tipo then VACIO else ERROR_TIPO
10	E.tipo := buscaTipo (num.entrada)
11	E.tipo := buscaTipo (num.num.entrada)

2.8 Manejo de errores semánticos

Libro: Compiladores. Conceptos fundamentales
 Capitulo: 6
 Tema: 6.5 Errores semánticos, pag. 126

2.9 Caso de estudio

Comprobador de Tipos de Lenguaje PROGRA

Las acciones semánticas se representan entre llaves de la forma {n}, donde n es el número del bloque en la tabla de acciones que se presenta posterior a la gramática.

```

P -> D C {1}
D -> V ; D {2} | ε {3}
V -> id : T {4}
T -> caracter {5} | entero {6} | real {7} | &T {8} | arreglo [ num ] of T {9}
C -> inicio S fin {10}
S -> Z ; S {11} | ε {12}
Z -> id {13} A {14} := E {15}
Z -> I L {16}
I -> si B entonces inicio S fin {17}
L -> sino inicio S fin {18} | ε {19}
Z -> mientras B hacer inicio S fin {20}
E -> H {21} R {22}
R -> oparit H {23} R {24} | ε {25}
H -> ( E ) {26} | -E {27} | id {28} A {29} | num {30} | num.num {31} | literal {32}
A -> [ E ] {33} | & {34} | ε {35}
B -> X {36} W {37}
W -> O X {38} W {39} | Y X {40} W {41} | ε {42}
X -> ( X ) {43} | id {44} oprel F {45} | cierto {46} | falso {47}
F -> id {48} A {49} | num {50} | num.num {51} | literal {52}

```

TABLA DE ACCIONES SEMANTICAS

Acción Semántica	Código de la Acción Semántica	# Error de Tipos
1	P.tipo := if D.tipo = vacio AND C.tipo = vacio then vacio else error tipo;	1
2	D.tipo := if V.tipo = vacio AND D.tipo = vacio then vacio else error tipo;	2
3	D.tipo := vacio;	
4	añadetipo (id.entrada, T.tipo); V.tipo := vacio;	
5	T.tipo := char;	
6	T.tipo := integer;	
7	T.tipo := real;	
8	T.tipo := pointer (T ₁ .tipo);	
9	T.tipo := array (num, T ₁ .tipo);	
10	C.tipo := S.tipo;	
11	S.tipo := if Z.tipo = vacio AND S.tipo = vacio then vacio else error tipo;	3
12	S.tipo := vacio;	
13	A.h := buscatipo (id.entrada);	
14	E.h := A.tipo;	
15	Z.tipo := if E.h = E.tipo then vacio else if E.h = real AND E.tipo = integer then vacio else error tipo;	4
16	Z.tipo := if I.tipo = vacio AND L.tipo = vacio then vacio else error tipo;	5

17	I.tipo := if B.tipo = boolean then S.tipo else error_tipo;	6
18	L.tipo := S.tipo;	
19	L.tipo := vacio;	
20	Z.tipo := if B.tipo = boolean then S.tipo else error_tipo;	7
21	R.h := H.tipo;	
22	E.tipo := R.tipo;	
23	<pre> R₁.h := if R.h = char OR H.tipo = char then if R.h = char AND H.tipo = char AND oparit.lexema = '+' then char else error_tipo; else if R.h = H.tipo then H.tipo else if R.h = real OR H.tipo = real then real else error_tipo; </pre>	8 9
24	R.tipo := R ₁ .tipo;	
25	R.tipo := R.h;	
26	H.tipo := E.tipo;	
27	H.tipo := E.tipo;	
28	A.h := buscatipo (id.entrada);	
29	H.tipo := A.tipo;	
30	H.tipo := integer;	
31	H.tipo := real;	
32	H.tipo := char;	
33	A.tipo := if E.tipo = integer AND A.h = array (s, t) then t else error_tipo;	10
34	A.tipo := if A.h = pointer (t) then t else error_tipo;	11
35	A.tipo := A.h;	
36	W.h := X.tipo;	
37	B.tipo := W.tipo;	
38	W ₁ .h := if W.h = boolean AND X.tipo = boolean then boolean else error_tipo;	12
39	W.tipo := W ₁ .tipo;	
40	W ₁ .h := if W.h = boolean AND X.tipo = boolean then boolean else error_tipo;	13
41	W.tipo := W ₁ .tipo;	
42	W.tipo := W.h;	
43	X.tipo := X ₁ .tipo;	
44	F.h := buscatipo (id.entrada);	
45	X.tipo := if F.h = F.tipo then boolean else error_tipo;	14
46	X.tipo := boolean;	
47	X.tipo := boolean;	
48	A.h := buscatipo (id.entrada);	
49	F.tipo := A.tipo;	
50	F.tipo := integer;	
51	F.tipo := real;	
52	F.tipo := char;	

Ejercicios: Analizar los siguientes programas escritos en lenguaje PROGRA hasta la etapa de análisis semántico.

1.

```
x : carácter ;
y : entero ;
z : arreglo [ 10 ] de entero ;

inicio
  x := "hola inges" ;
  mientras y < 10 hacer
    inicio
      z [ y ] := 0;
    fin;
  fin.
fin.
```

2.

```
x : real ;
y : entero ;

inicio
  si x > 3.14 Y y = 0 entonces
    inicio
      y := x + 1;
    fin;
  fin.
fin.
```